

COMP 8005

Assignment 3

Design

Manraj Bhullar

A01018456

February 20, 2026

Overview

The purpose of this design document is to provide details of the next iteration of the distributed UNIX password cracking system. The system has evolved from the previous single-worker architecture into a fully distributed multi-worker system where multiple hosts cooperate to search the password space. In the prior iteration the controller assigned one full cracking job to a worker which handled the entire search space on a single machine using threads.

In this version the search space is globally partitioned and workers dynamically request work from the controller in real time. Each worker receives only a portion of the search space and repeatedly requests additional chunks after finishing its assigned range. The controller manages global termination so that once any worker discovers the correct password, no new work is issued and all workers are instructed to stop.

Controller Design

Goal

The goal of the controller is to coordinate a distributed password cracking operation across multiple workers. It must parse the shadow entry, divide the global search space into chunks, assign work dynamically, monitor worker progress, and enforce a global stop condition once the password is found.

Success

The controller is considered successful when it correctly registers multiple workers, distributes unique chunks of work without duplication, and stops all workers once a valid password is discovered. It must also report final timing measurements and performance statistics.

Invariant

The controller must never assign the same chunk twice. Each chunk must be issued at most once. Once the password is found, no additional chunks may be assigned and all workers must be instructed to stop.

Controller Responsibilities

1. Accepting a shadow file, username, port, heartbeat interval, and chunk size from the command line.
2. Parsing the shadow file to extract the hashing algorithm identifier, salt, and password hash.
3. Accepting multiple worker registrations.
4. Maintaining a global chunk allocator for the search space.
5. Responding to worker chunk requests with unique disjoint ranges.

6. Receiving periodic heartbeat responses from workers.
7. Receiving completion notifications and final results.
8. Enforcing a global stop when any worker finds the password.
9. Reporting timing breakdowns and final results.

Testing Strategy

Test 1: Chunk Allocation Correctness

Steps

1. Run multiple workers simultaneously.
2. Log assigned chunk ranges for each worker.

Expected Result

- No chunk is assigned twice.
- Workers never test overlapping candidate ranges.

What it Proves

- The global allocator works correctly and prevents duplicate work.

Test 2: Global Stop Condition

Steps

1. Run several workers on a known password.
2. Observe system behavior once password is found.

Expected Result

- Controller stops issuing chunks.
- Workers terminate after receiving STOP message.

What it Proves

- The distributed termination protocol works correctly.

Test 3: Multiple Worker Registration

Steps

1. Run the system with different numbers of workers.
2. Observe effects under certain test scenarios to ensure correctness.

Expected Result

- Workers continuously request work after finishing chunks.
- The system scales naturally with worker count.

What it Proves

- Worker registration and workload distribution functions correctly.

Controller FSM

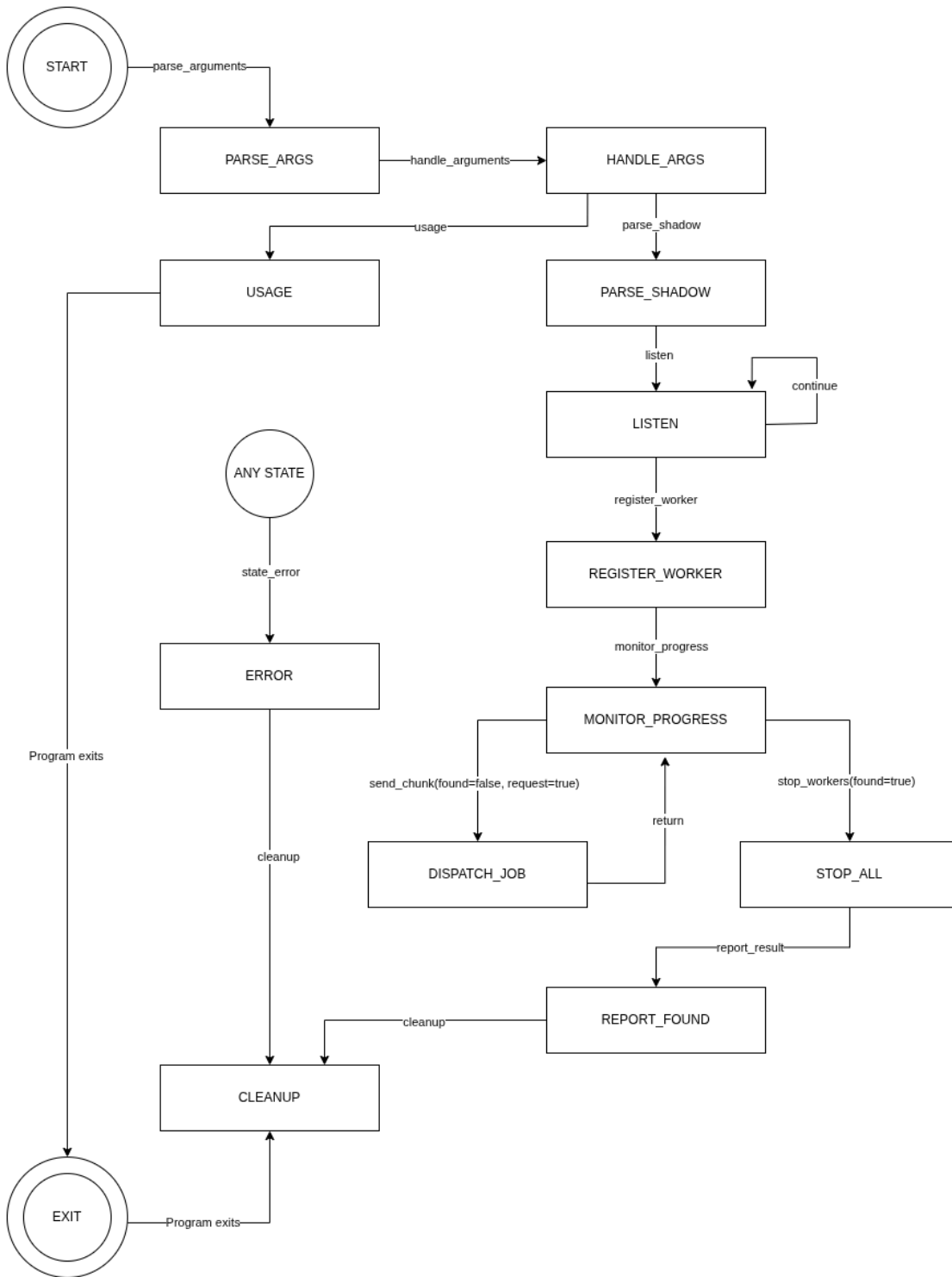
State Meanings

State	Meaning
START	Program is ran.
PARSE_ARGS	Command-line arguments get parsed.
HANDLE_ARGS	Parsed arguments are validated and prepared for use.
PARSE_SHADOW	Shadow file is read and the username's entry is parsed to extract algorithm, salt, and hash.
LISTEN	Opens connection socket and continually accepts new worker connections.
REGISTER_WORKER	A worker joins the distributed cracking session.
MONITOR_PROGRESS	Tracks all active workers and receives updates.
DISPATCH_JOB	Assigns a portion of the search space to a worker requesting a new chunk.
STOP_ALL	Stops all workers after the password is found.
REPORT_FOUND	Outputs final cracking results.
ERROR	Reports when a state error occurs and redirects to cleanup.
CLEANUP	Program releases resources and closes connections.
USAGE	Displays help message when program input is invalid.
EXIT	Program exits.

State Transition Table

Current State	Call (function)	Fail State	Success State
START	parse_arguments		PARSE_ARGS
PARSE_ARGS	handle_arguments	USAGE	HANDLE_ARGS
HANDLE_ARGS	parse_shadow	USAGE	PARSE_SHADOW
PARSE_SHADOW	listen	ERROR	LISTEN
LISTEN	register_worker	ERROR	MONITOR_PROGRESS
MONITOR_PROGRESS	send_chunk stop_workers	ERROR ERROR	DISPATCH_JOB STOP_ALL
STOP_ALL	report_result	ERROR	REPORT_FOUND
REPORT_FOUND	cleanup	ERROR	CLEANUP
ERROR	cleanup		CLEANUP
CLEANUP	Program exits		EXIT
USAGE	Program exits		EXIT

FSM Diagram



Functions

Name	Inputs	Outputs	Responsibility
parse_arguments	Command-line inputs	Parsed arguments	Parse command-line arguments for shadow file, username, port, heartbeat interval, and chunk size.
handle_arguments	Parsed arguments	Validated arguments	Validate arguments and ensure values are usable before program execution.
parse_shadow	Shadow file path and username	Password data	Read the shadow file and extract the selected user's password information.
listen	Port number	Controller socket	Initialize the controller server socket and accept worker connections.
register_worker	Worker connection	Registration status	Accept and record a worker registration so it can participate in cracking.
monitor_progress	Active worker connections	Worker progress information	Manage all active workers, receive heartbeat updates, and detect chunk completion or password discovery.

send_chunk	Worker request and global allocator	Assigned job	Assign the next available portion of the search space to a requesting worker.
stop_workers	Found password flag	Stop flag set	Notify all workers to terminate once the password has been discovered.
report_result	Final cracking result	Printed cracking results	Display final result and timing breakdown.
state_error	Error context	Error message	Handle runtime state errors and transition to cleanup.
cleanup	Error message	Exit status	Close sockets and terminate the controller cleanly.
usage	Program context	Usage message	Print usage instructions when arguments are invalid.

Worker Design

Goal

The goal of the worker is to repeatedly request work from the controller and crack only its assigned portion of the search space using multiple threads.

Success

The worker is successful when it correctly requests chunks, attempts to crack each without duplication, responds to heartbeats, reports results, and stops immediately when instructed or any worker finds the correct password.

Invariant

The worker must never attempt candidates outside its assigned chunk. Each candidate inside the chunk must be tested exactly once across all threads.

Worker Responsibilities

1. Register with the controller.
2. Request a chunk of work.
3. Crack assigned chunk using T threads.
4. Report FOUND immediately if discovered.
5. Request next chunk if exhausted.
6. Respond to heartbeat requests.
7. Exit when STOP received.

Testing Strategy

Test 1: Chunk Request Validation

Steps

1. Start the worker and controller normally.
2. Observe worker behavior after finishing a chunk.

Expected Result

- The worker continuously requests new work after completing each chunk if the password has still not been found by the system.

What it Proves

- The worker correctly handles cracking using the chunk model.

Test 2: Stop Signal Handling

Steps

1. Use a password early in the search space and run multiple workers.
2. Observe worker behavior when one worker finds the password.

Expected Result

- The worker that finds the password reports FOUND immediately.
- Other workers stop requesting chunks and exit cleanly.

What it Proves

- Workers correctly follow the global termination protocol.

Test 3: Multithreading Within a Chunk

Steps

1. Run the system using different thread counts with the same chunk size.
2. Compare completion behaviour.

Expected Result

- Higher thread counts reduce runtime to crack a chunk.

What it Proves

- Thread partitioning remains correct throughout cracking of each chunk.

Worker FSM

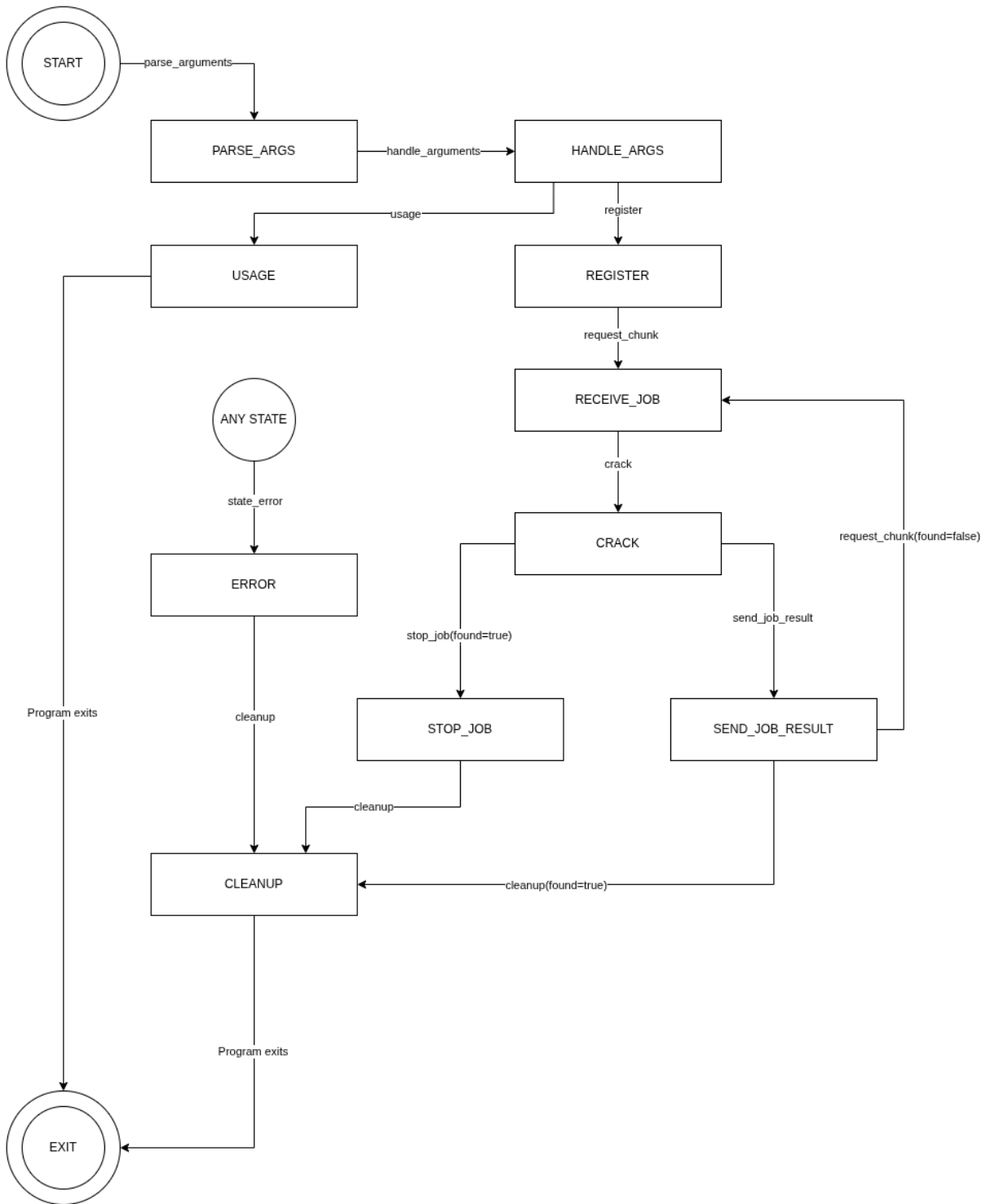
State Meanings

State	Meaning
START	Program is ran.
PARSE_ARGS	Command-line arguments get parsed.
HANDLE_ARGS	Parsed arguments are validated and prepared for use.
REGISTER	Worker connects and registers with the controller.
RECEIVE_JOB	Worker receives next job parameters from controller.
CRACK	Cracking process for currently assigned job/chunk.
STOP_WORKER	Worker stops if the password has been found by another worker.
SEND_JOB_RESULT	Worker sends results from current cracking job that has finished.
ERROR	Reports when a state error occurs and redirects to cleanup.
CLEANUP	Program releases resources and closes connections.
USAGE	Displays help message when program input is invalid.
EXIT	Program exits.

State Transition Table

Current State	Call (function)	Fail State	Success State
START	parse_arguments		PARSE_ARGS
PARSE_ARGS	handle_arguments	USAGE	HANDLE_ARGS
HANDLE_ARGS	register	USAGE	REGISTER
REGISTER	request_chunk	ERROR	RECEIVE_JOB
RECEIVE_JOB	crack	ERROR	CRACK
CRACK	stop_job	ERROR	STOP_JOB
	send_job_result	ERROR	SEND_JOB_RESULT
STOP_JOB	cleanup	ERROR	CLEANUP
SEND_JOB_RESULT	request_chunk	ERROR	RECEIVE_JOB
	cleanup	ERROR	CLEANUP
ERROR	state_error		CLEANUP
CLEANUP	Program exits		EXIT
USAGE	Program exits		EXIT

FSM Diagram



Functions

Name	Inputs	Outputs	Responsibility
parse_arguments	Command-line inputs	Parsed arguments	Parse command-line arguments for shadow file, username, port, heartbeat interval, and chunk size.
handle_arguments	Parsed arguments	Validated arguments	Validate arguments and ensure values are usable before program execution.
register	Connection details	Registration status	Connect to the controller and register the worker.
request_chunk	Controller connection	Assigned job details	Request a new portion of the search space from the controller.
crack	Assigned job details and thread count	Progress updates and job results	Perform password cracking using multiple threads on the assigned chunk.
stop_job	Stop signal from controller	Stop confirmation	Process cracking job while responding to heartbeat requests from controller.
send_job_result	Job results data	Result sent status	Send the discovered password or

			exhaustion status to the controller.
state_error	Error context	Error message	Handle runtime state errors and transition to cleanup.
cleanup	Error message	Exit status	Close sockets and terminate the controller cleanly.
usage	Program context	Usage message	Print usage instructions when arguments are invalid.